

Exerciții — desene geometrice

O ierarhie de clase construită doar cu moștenire și polimorfism · Java · Adrian

Ce ai voie să folosești: clase, câmpuri `private / protected`, `getter` și `setter`i, mai mulți constructori pe aceeași clasă, `extends`, `super(...)` și `super.metoda()`, `@Override`, `instanceof`, `cast`, `List + ArrayList`, `for-each`, `if/else`, `System.out.println`.

Încă NU folosim: `abstract`, `interface`, `toString()`, `equals / hashCode`, `clone()`, `static`, fișiere, `try/catch`, `stream-uri`, desenare grafică. Totul se „afișează” cu `System.out.println`.

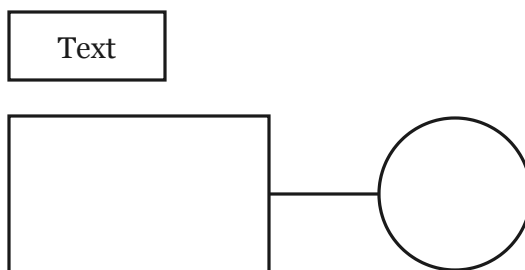
Unde lucrezi: proiect nou, pachetul `app.figuri`. Coordonatele sunt numere întregi.

Enunțul

Să se definească o ierarhie de clase pentru un modul care are ca scop **afișarea** și **manipularea** unor **desene geometrice**. Un desen geometric este format din **zero sau mai multe elemente**. Elementele sunt figuri geometrice din setul:

Element	Definire
Punct	Coordonatele x, y
Linie	Două puncte
Cerc	Un punct și raza
Dreptunghi	Două puncte
Etichetă	Două puncte ce definesc un dreptunghi și textul ce trebuie scris în interiorul dreptunghiului

Exemplu de desen geometric care trebuie să poată fi afișat și manipulat:



o etichetă, un dreptunghi, o linie și un cerc — patru elemente într-un singur desen

Afișare înseamnă prezentarea pe ecran a desenului geometric.

Manipulare înseamnă:

- **Translatare** (mutare) pe orizontală și/sau verticală a desenului geometric — modificarea x-ului / y-ului tuturor elementelor cu o valoare dată;
- **Duplicare** desen geometric — crearea unui desen nou care să aibă aceleași componente la aceleași coordonate.

Cerința 1 (pe hârtie, înainte de orice linie de cod): desenează arborele de clase — cine extinde pe cine, ce câmpuri are fiecare clasă și ce metode suprascrie. Abia după ce l-ai desenat treci la exercițiul 1. La final compari hârtia cu codul.

Partea 1 — ierarhia

1. Rădăcina: **Figura** și prima subclasă, **Punct**

ușor

Scrie clasa **Figura** cu exact trei metode:

```
public String descriere()           // returneaza "FIGURA"
public void muta(int dx, int dy)    // corp gol
public void afiseaza()              // System.out.println(descriere())
```

Apoi **Punct** extends **Figura**: câmpuri private **x** și **y**, constructor cu doi parametri, getteri și setteri.

Suprascrie **descriere()** și **muta(...)**. Pe **afiseaza()** **nu ai voie** să o suprascrii în nicio figură — nici în **Punct**, nici în vreuna dintre cele care urmează.

```
Punct p = new Punct(3, 4);
p.afiseaza();
p.muta(10, 0);
p.afiseaza();
```

```
PUNCT (3,4)
PUNCT (13,4)
```

2. **Linie**, **Cerc**, **Dreptunghi**

ușor

Toate trei extind **Figura** și toate trei sunt construite din **Punct**, nu din numere:

- **Linie** — două puncte;
- **Cerc** — un punct (centrul) și raza (int);
- **Dreptunghi** — două puncte (colțul stânga-jos și colțul dreapta-sus).

Fiecare suprascrie **descriere()** și **muta(...)**. Formatul de afișare e al tău, dar ține-l pe cel de mai jos ca să putem compara.

```
LINIE (0,0)-(10,0)
CERC centru (20,5), raza 3
DREPTUNGHI (2,2)-(12,8)
```

Regula exercițiului: în **muta(...)** la **Linie**, **Cerc** și **Dreptunghi** nu ai voie să atingi direct **x** sau **y**. Singura clasă care știe să adune **dx** la un **x** e **Punct**.

*Dacă un **Cerc** e format dintr-un **Punct**, iar **Punct** știe deja să se mute, cât cod mai are de scris **Cerc** ca să se mute?*

3. Eticheta — subclasa care nu mai scrie nimic

ușor

Din enunț: o etichetă e „două puncte ce definesc un dreptunghi și textul ce trebuie scris în interiorul dreptunghiului”. Scrie `Eticheta` așa încât să adauge doar câmpul `text` și să suprascrie doar `descriere()`.

Regula exercițiului: în `Eticheta` nu ai voie să scrii metoda `muta(...)` și nu ai voie să declari câmpuri de tip `Punct`.

```
Eticheta e = new Eticheta(new Punct(2, 9), new Punct(8, 11), "Text");
e.afiseaza();
e.muta(0, -5);
e.afiseaza();
```

```
ETICHETA (2,9)–(8,11) "Text"
ETICHETA (2,4)–(8,6) "Text"
```

Eticheta s-a mutat corect fără ca tu să scrii o linie de mutare pentru ea. Explică în două rânduri, în scris, de unde a venit codul care a rulat.

Partea 2 — polimorfismul la lucru

4. Desen — zero sau mai multe elemente

mai greu

Clasa `Desen` ține o `List<Figura>` și are:

- `public void adauga(Figura f);`
- `public void afiseaza()` — afișează toate elementele;
- `public void muta(int dx, int dy)` — translatează tot desenul.

Regula exercițiului: în cele două bucle nu ai voie să scrii niciun `if`, niciun `instanceof` și niciun `cast`. Dacă ai nevoie de ele, ceva lipsește din `Figura` — întoarce-te la exercițiul 1.

Construiește desenul din enunț — eticheta, dreptunghiul, linia și cercul — și translatează-l cu `(100, 0)`:

```
--- desenul initial ---
ETICHETA (2,9)–(8,11) "Text"
DREPTUNGHI (2,2)–(12,8)
LINIE (12,5)–(17,5)
CERC centru (20,5), raza 3
--- dupa muta(100, 0) ---
ETICHETA (102,9)–(108,11) "Text"
DREPTUNGHI (102,2)–(112,8)
LINIE (112,5)–(117,5)
CERC centru (120,5), raza 3
```

Un desen gol trebuie să meargă și el: `new Desen().afiseaza();` nu are voie să crape.

5. Duplicarea

mai greu

Adaugă în `Figura` metoda `public Figura copie()` și suprascrie-o în fiecare subclasă, ca fiecare figură să întoarcă un obiect nou **de clasa ei**, cu aceleași coordonate. Apoi scrie în `Desen` :

```
public Desen duplica()
```

care întoarce un desen nou, cu aceleași componente la aceleași coordonate. **Aceeași regulă:** în bucla din `duplica()` nu ai voie să scrii `if`, `instanceof` sau `cast`.

Testul care contează — duplică desenul, apoi mută **numai** originalul:

```
Desen copie = desen.duplica();
desen.muta(100, 0);
desen.afiseaza();
copie.afiseaza();
```

Dacă se mișcă amândouă, duplicarea ta e greșită chiar dacă programul rulează fără eroare. Scrie pe fișă ce ai văzut prima dată când ai rulat.

Un desen care conține aceleași obiecte nu e o copie; e același desen văzut de două ori. Ce anume ai pus în listă — obiecte sau adrese către ele?

6. Cerința 2 din enunț: schița de program

mai greu

Într-o clasă `App` scrie un program care, în ordinea asta:

- creează desenul din enunț și îl afișează;
- îl duplică;
- translatează duplicatul cu `(0, 20)`;
- afișează ambele desene, cu câte un titlu între ele;
- creează un al treilea desen, gol, îl afișează, apoi adaugă în el un singur punct și îl afișează din nou.

Tot programul trebuie să încapă fără niciun `instanceof` și fără niciun `cast`. Dacă ai scris unul, notează pe fișă unde și de ce ai simțit nevoia.

7. Un desen care conține un desen

provocare

Clientul cere acum desene compuse: un desen să poată conține, pe lângă figuri, și alte desene — iar mutarea și afișarea să meargă la fel, până în adâncime.

Regula exercițiului: pentru afișare și mutare ai voie să modifici o singură linie din `Desen` — antetul clasei — și nimic altceva. Nicio buclă nu se schimbă, niciun `if` nu apare.

```
Desen interior = new Desen();
interior.adauga(new Cerc(new Punct(20, 5), 3));

Desen exterior = new Desen();
exterior.adauga(new Dreptunghi(new Punct(0, 0), new Punct(30, 30)));
exterior.adauga(interior);

exterior.muta(5, 5);
exterior.afiseaza();
```

Cercul trebuie să ajungă la `(25,10)`, deși nimeni nu l-a mutat direct. Explică în scris, în trei rânduri, lanțul de apeluri prin care a ajuns acolo.

Apoi verifică și duplicarea: cheamă `duplica()` pe desenul exterior și afișează rezultatul. Nu iese ce te aștepți. Găsești singur ce lipsește și scrii pe fișă ce anume — nu e nevoie să repari.

8. Moștenirea care compilează și tot e greșită

provocare

`Linie` are două puncte. `Dreptunghi` are tot două puncte. Tentația e să scrii `Dreptunghi extends Linie` și să scapi de câmpuri. Fă-o — pe o copie a proiectului, nu peste ce ai lucrat. Compilează și rulează.

Apoi adaugă în `Linie` metoda `public double lungime()` (distanța dintre cele două puncte) și rulează:

```
Linie l = new Dreptunghi(new Punct(0, 0), new Punct(3, 4));
System.out.println(l.lungime());
```

Răspunde în scris, la fiecare:

- ce număr a ieșit și ce reprezintă el, de fapt, pentru un dreptunghi;
- de ce compilatorul n-a avut nimic de obiectat;
- întrebarea pe care trebuia să ți-o pui **înainte** de `extends` — și care e răspunsul ei pentru perechea `Dreptunghi / Linie`, respectiv pentru perechea `Eticheta / Dreptunghi` de la exercițiul 3.

La final aruncă copia și păstrează varianta corectă.